

**FEATURE DRIFT DETECTION VIA ADVERSARIAL
VALIDATION**

NUSRAT BEGUM

**A THESIS SUBMITTED IN PARTIAL FULFILLMENT
OF THE REQUIREMENTS FOR THE DEGREE OF
MASTER OF SCIENCE (COMPUTER SCIENCE)
FACULTY OF GRADUATE STUDIES
MAHIDOL UNIVERSITY
2026**

COPYRIGHT OF MAHIDOL UNIVERSITY

Thesis
entitled
**FEATURE DRIFT DETECTION VIA ADVERSARIAL
VALIDATION**

.....
Ms. Nusrat Begum
Candidate

.....
Asst. Prof. Thanapon Noraset,
Ph.D. (Computer Science)
Major advisor

.....
Assoc. Prof. Suppawong Tuarob,
Ph.D. (Computer Science)
Co-advisor

.....
Assistant Professor Dr. Warin
Krityakiarana
Deputy Dean for Planning and Quality
Assurance
Acting as Dean of the Faculty of Graduate
Studies, Mahidol University
Dean
Faculty of Graduate Studies
Mahidol University

.....
Assoc. Prof. Boonsit Yimwadsana,
Ph.D. (Electrical Engineering)
Program Director
Master of Science Programme
in Computer Science
Faculty of Information and
Communication Technology
Mahidol University

Thesis
entitled
**FEATURE DRIFT DETECTION VIA ADVERSARIAL
VALIDATION**

was submitted to the Faculty of Graduate Studies, Mahidol University
for the degree of Master of Science (Computer Science)

on
February 2026

.....
Ms. Nusrat Begum
Candidate

.....
Assistant Professor Dr. Akara Supratak,
Ph.D. (Computer Science)
Chair

.....
Asst. Prof. Thanapon Noraset,
Ph.D. (Computer Science)
Member

.....
Assoc. Prof. Suppawong Tuarob,
Ph.D. (Computer Science)
Member

.....
Assistant Professor Dr. Warin
Krityakiarana
Deputy Dean for Planning and Quality
Assurance
Acting as Dean of the Faculty of Graduate
Studies, Mahidol University
Dean
Faculty of Graduate Studies
Mahidol University

.....
Dr. Pattanasak Mongkolwat,
Ph.D. (Computer Science)
Dean
Faculty of Information and
Communication Technology
Mahidol University

ACKNOWLEDGEMENTS

Nusrat Begum

FEATURE DRIFT DETECTION VIA ADVERSARIAL VALIDATION

NUSRAT BEGUM 6638009 ITCS/M

M.Sc. (COMPUTER SCIENCE)

THESIS ADVISORY COMMITTEE: THANAPON NORASET, Ph.D., SUPPAWONG TUAROB, Ph.D.

ABSTRACT

Machine learning models deployed in production environments frequently experience distribution drift—changes in input feature distributions over time that can degrade model performance. While discriminative methods like D3 have proven that adversarial validation can accurately detect drift, they suffer from the “black box” problem—providing no insight into the source of the drift. In complex MLOps pipelines, knowing *that* drift occurred is insufficient; engineers must understand *what* drifted to take appropriate corrective action.

This thesis proposes **Explainable Adversarial Drift Detection (EADD)**, a novel framework that extends adversarial validation with SHAP-based root cause analysis. Unlike univariate methods (e.g., KSWIN) that flood users with alerts for every feature, EADD employs a multivariate permutation test to robustly flag significant shifts, followed by an automated diagnostic step that ranks features by their contribution to the drift. This transforms the drift detector from a simple alarm into an intelligent diagnostic tool.

The main contributions are: (1) a streaming adversarial validation framework with reservoir sampling for representative reference windows and statistical rigor through permutation testing; (2) SHAP-based root cause analysis that identifies which specific features drive detected drift; (3) automated drift prescriptions providing actionable remediation recommendations based on drift patterns (univariate, multivariate, or feature-subset drift); (4) comprehensive benchmarking against state-of-the-art methods (D3, ADWIN, KSWIN, MMD) on the Lukats et al. (2025) real-world data streams; and (5) practical implementation guidelines for MLOps deployment including threshold calibration and computational efficiency considerations.

This work establishes EADD as a principled, explainable drift detection methodology that advances beyond detection accuracy to actionable diagnostics, addressing the critical explainability gap identified in recent benchmarks.

IMPLICATION OF THE THESIS

EADD provides both high detection accuracy and actionable explanations, enabling MLOps teams to respond appropriately to different drift scenarios rather than treating all drift signals as equivalent alarms requiring full model retraining.

KEY WORDS : DRIFT DETECTION / ADVERSARIAL VALIDATION /
COVARIATE DRIFT / MLOPS / DISTRIBUTION SHIFT /
STREAMING DATA

56 pages

CONTENTS

	Page
ACKNOWLEDGEMENTS	iii
ABSTRACT	iv
LIST OF TABLES	ix
LIST OF FIGURES	x
CHAPTER I INTRODUCTION	1
1.1 Motivation	1
1.2 Objectives	2
1.3 Expected Contributions	3
CHAPTER II LITERATURE REVIEW	5
2.1 Overview	5
2.2 Types of Drift	6
2.2.1 Drift by Nature of Change	6
2.2.2 Drift by Temporal Pattern	7
2.3 Classification of Drift Detection Methods	7
2.3.1 Supervised Drift Detection Methods	8
2.3.2 Unsupervised Drift Detection Methods	9
2.3.3 Hybrid and Ensemble Methods	14
2.4 Limitations of Existing Methods	15
2.5 Related Work: Adversarial Validation	16
2.5.1 Concept and Origins	16
2.5.2 Industrial Applications	16
2.6 Gap in the Literature	17
CHAPTER III METHODOLOGY	19
3.1 Overview: Explainable Adversarial Drift Detection (EADD)	19
3.2 Problem Formulation	20
3.3 EADD Framework Components	21

CONTENTS (cont.)

	Page
3.3.1 Step 1: Adaptive Reference Windowing	22
3.3.2 Step 2: Adversarial Validation (The Detector)	22
3.3.3 Step 3: Statistical Significance via Permutation Test	23
3.3.4 Step 4: Feature Drift Explainability via SHAP (Novel Contribution)	24
3.3.5 Automated Drift Prescription (Novel Contribution)	25
3.3.6 Model Adaptation Strategies	26
3.4 Datasets	27
3.5 Baseline Drift Detectors	28
3.6 Evaluation Metrics	28
3.7 Experimental Design	29
3.8 Practical Considerations	30
CHAPTER IV EXPERIMENTAL DESIGN	31
4.1 Experimental Setup	31
4.2 Experiment 1: Sensitivity to Temporal Drift Patterns	32
4.3 Experiment 2: Real-World Benchmark	33
4.4 Experiment 3: Explainability Case Study	34
4.5 Experiment 4: Robustness to Stable Data (False Alarm Evaluation)	36
4.6 Summary of Experimental Design	37
CHAPTER V DISCUSSION	38
5.1 Practical Implementation Considerations	38
5.1.1 Defining Reference and Current Windows	38
5.1.2 Window Size and Step Size Selection	39
5.1.3 AUC Threshold Selection	40
5.2 Addressing Core Research Questions	40
5.2.1 RQ1: How Does EADD Handle “Unknown” or “Real” Data?	40
5.2.2 RQ2: How Do We Distinguish Drift Types?	41

CONTENTS (cont.)

	Page
5.2.3 RQ3: How Is EADD Better Than Existing Methods?	42
5.3 MLOps Prescription Framework	42
5.4 Limitations	42
5.5 Chapter Summary	43
CHAPTER VI CONCLUSION	44
6.1 Summary of Contributions	44
6.2 Implications for MLOps	45
6.3 Limitations	45
6.4 Future Work	46
6.5 Concluding Remarks	46
APPENDIX	48
APPENDIX A IMPLEMENTATION DETAILS	49
A.1 Adversarial Classifier (LightGBM)	49
A.2 Windowing Strategy	49
A.3 Permutation Test	50
A.4 SHAP Configuration	50
A.5 Computational Environment	50
BIOGRAPHY	56

LIST OF TABLES

Table	Page
2.1 Comparison of Drift Detection Approaches	10
3.1 Alignment of EADD Components with Research Requirements	21
4.1 Summary of Experimental Design	37
5.1 Guidelines for Window Size Selection	39
5.2 MLOps Prescription Framework Based on Drift Diagnosis	43
A.1 LightGBM Hyperparameters	49

LIST OF FIGURES

Figure	Page
2.1 Comparison of no drift versus drift scenarios. Left: When distributions overlap, the adversarial classifier cannot distinguish data sources ($AUC \approx 0.5$). Right: When drift occurs, distributions shift and the classifier can distinguish sources ($AUC \gg 0.5$).	5
3.1 Overview of Explainable Adversarial Drift Detection (EADD) methodology. Blue boxes indicate novel contributions: Permutation Test (Step 3) and SHAP Feature Attribution (Step 4). The framework extends D3's adversarial validation with statistical rigor and root cause analysis.	19

CHAPTER I

INTRODUCTION

1.1 Motivation

Machine learning models deployed in real-world environments frequently encounter evolving data distributions over time. According to Lu et al. (2018) and Gama et al. (2014), these changes—collectively referred to as distribution shift or drift—can manifest as changes in input features (covariate drift) or in the relationship between inputs and labels (concept drift), causing model performance to deteriorate if left unmanaged. Lu et al. (2018) and Widmer and Kubat (1996) emphasize that detecting such changes early is essential for maintaining reliable model performance, reducing operational risk, and avoiding accumulating technical debt in production systems (Bayram et al., 2022; Sculley et al., 2015).

Conventional model monitoring strategies react to drift only after observable performance degradation, which by design means that the system has already suffered from poor predictions on new data (Widmer and Kubat, 1996). In production contexts where ground-truth labels may be delayed, unavailable, or costly to obtain, this post-hoc reactionary approach is often inadequate (Lu et al., 2018; Souza et al., 2020a). According to Lu et al. (2018) and Bayram et al. (2022), early drift detection that anticipates distribution changes before significant performance loss is therefore a practical priority in dynamic machine learning systems.

Current drift detection methods are varied. Traditional statistical methods employ hypothesis tests—such as the Kolmogorov–Smirnov test, chi-squared test, or Kullback–Leibler divergence—to detect distribution changes by comparing incoming and reference data (Rabanser et al., 2019; Raab et al., 2020). Bifet and Gavaldà (2007) developed adaptive windowing methods like ADWIN that dynamically adjust how much historical data is retained for comparison. Supervised error-based detectors like DDM and EDDM monitor model performance metrics over time, but require access to ground-truth

labels (Gama et al., 2004; Baena-García et al., 2006). Prediction-based detectors track model outputs for changes in confidence, entropy, or prediction distributions without necessarily requiring labels (Sethi and Kantardzic, 2017). Despite this diversity, several challenges remain: statistical tests can be sensitive to noise and may struggle in high-dimensional settings; supervised methods depend on label availability; prediction-based methods may be confounded by model calibration issues; and hybrid approaches often require engineered context variables that may not generalize across domains (Lu et al., 2018; Webb et al., 2016).

To address these gaps, we propose recasting feature drift detection as a discriminative learning problem: treating the comparison between historical and newly arriving data as a binary classification task. If an auxiliary classifier can reliably separate data from the two time periods based solely on features, it indicates a meaningful distributional change. This approach, known as adversarial validation, has been used in data science to detect and quantify dataset shift Pan et al. (2020); Qian and Rao (2021) but has not been systematically developed into a general feature drift detector for streaming or window-based data monitoring.

According to Lopez-Paz and Oquab (2017), this approach offers several key advantages for drift detection: it is label-independent (does not require ground-truth labels for new data), model-agnostic (can be applied regardless of the primary prediction model), makes no assumptions about which features or contexts are important, and can detect complex high-dimensional, non-linear distributional shifts that may evade univariate or marginal statistical tests.

1.2 Objectives

The primary objective of this thesis is to develop **Explainable Adversarial Drift Detection (EADD)**, a novel framework that extends adversarial validation with root cause analysis and actionable prescriptions. Specifically, this work aims to:

1. Formalize feature drift detection as a discriminative two-sample learning problem, extending beyond binary detection to provide **feature-level**

attribution of drift sources using SHAP-based interpretability methods (Lundberg and Lee, 2017).

2. Develop streaming and sliding window procedures with **reservoir sampling** for representative reference windows, and **permutation testing** for statistically principled threshold calibration.
3. Systematically benchmark EADD against state-of-the-art unsupervised drift detectors—including D3 Gözüaık et al. (2019), ADWIN Bifet and Gavaldà (2007), KSWIN Raab et al. (2020), and MMD Gretton et al. (2012)—on the real-world datasets from the Lukats et al. (2025) benchmark Lukats et al. (2025).
4. Design and validate an **automated prescription system** that provides actionable remediation recommendations based on drift diagnosis (univariate drift, multivariate concept shift, or feature-subset drift).
5. Evaluate practical applicability in MLOps settings, including computational efficiency, threshold sensitivity, and integration with model retraining strategies.

1.3 Expected Contributions

This thesis seeks to contribute:

1. **EADD Framework:** A formal explainable adversarial validation framework that combines discriminative drift detection with SHAP-based root cause analysis, transforming drift detection from a binary alarm into an intelligent diagnostic tool.
2. **Root Cause Analysis:** SHAP-based feature importance extraction that identifies which specific features drive detected drift, addressing the “black box” limitation of existing methods like D3 (Gözüaık et al., 2019).

3. **Automated Prescriptions:** A novel classification of drift patterns (univariate, multivariate, feature-subset) with corresponding actionable remediation recommendations for MLOps teams.
4. **Benchmark Evaluation:** Comprehensive comparison with state-of-the-art unsupervised detectors on real-world data streams, demonstrating both detection accuracy and the value of explainability.
5. **Practical Guidelines:** Deployment recommendations for MLOps pipelines, including permutation-based threshold calibration, reservoir sampling for reference windows, and computational efficiency considerations.

CHAPTER II

LITERATURE REVIEW

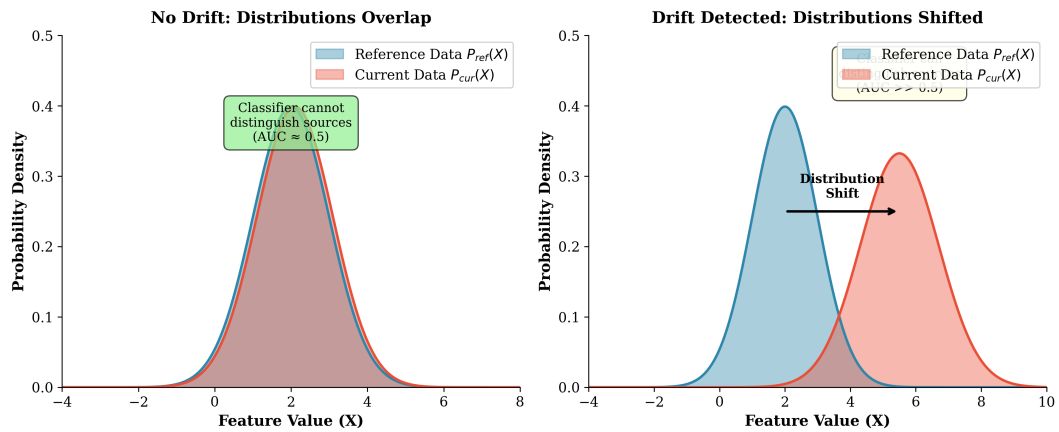


Figure 2.1: Comparison of no drift versus drift scenarios. Left: When distributions overlap, the adversarial classifier cannot distinguish data sources ($AUC \approx 0.5$). Right: When drift occurs, distributions shift and the classifier can distinguish sources ($AUC > 0.5$).

2.1 Overview

Drift detection addresses the challenge that arises when the statistical characteristics of data evolve over time in a manner that impacts the validity of a machine learning model (Lu et al., 2018; Gama et al., 2014). According to Bayram et al. (2022), when users, environments, data sources, or the relationships between labels and features change, model accuracy can diminish. Gama et al. (2014) and Sculley et al. (2015) emphasize that drift detection identifies this modification early, allowing practitioners to adapt, retrain, or update the model accordingly. A robust understanding of drift types and detection methods is essential for designing effective monitoring systems.

2.2 Types of Drift

Drift can be categorized along two main dimensions: by the nature of the change (what component of the data-generating process is affected) and by the temporal pattern of the change (how the drift manifests over time) (Lu et al., 2018; Gama et al., 2014).

2.2.1 Drift by Nature of Change

Based on which component of the joint distribution $P(X, y)$ changes, the literature (Lu et al., 2018; Gama et al., 2014; Tsymbal, 2004) classifies drift into the following types:

Concept Drift (Real Drift): The conditional distribution $P(y|X)$ changes over time while the feature distribution $P(X)$ may or may not change (Lu et al., 2018; Žliobaitė, 2010). This represents a change in the underlying relationship between inputs and outputs—the “concept” the model has learned becomes invalid. For example, the factors that determine creditworthiness may evolve due to economic shifts, even if the applicant characteristics (features) remain similar (Gama et al., 2014). Concept drift directly degrades model performance because the learned decision boundaries no longer reflect the true input-output relationship (Lu et al., 2018).

Covariate Drift (Feature Drift / Virtual Drift): The marginal distribution of input features $P(X)$ changes over time while the conditional relationship $P(y|X)$ remains stable (Tsymbal, 2004; Shimodaira, 2000). The ground-truth concept does not change, but the model may encounter inputs outside its training distribution. For example, a fraud detection model trained on transaction patterns from one demographic may receive data from a different user population, even though the definition of fraud remains unchanged (Tsymbal, 2004). Covariate drift can degrade model performance because the model is exposed to feature regions where it may not generalize well (Shimodaira, 2000).

Prior Probability Drift (Label Shift): The marginal distribution of labels $P(y)$ changes over time while $P(X|y)$ remains stable (Tsymbal, 2004). This occurs when the prevalence of different classes changes—for instance, a seasonal increase in a particular disease affecting the class balance in medical diagnosis. This is sometimes called class imbalance shift (Lu et al., 2018).

Note on Terminology: The literature uses varied terminology (Lu et al., 2018). The term “concept drift” is sometimes used as an umbrella term for any distribution change affecting model performance (Gama et al., 2014), while some reserve it specifically for changes in $P(y|X)$ (Tsymbal, 2004). This thesis focuses primarily on covariate (feature) drift detection, as it enables early warning without requiring labels.

2.2.2 Drift by Temporal Pattern

Based on how drift manifests over time, the literature (Lu et al., 2018; Gama et al., 2014; Žliobaitė, 2010) categorizes drift as follows:

- **Abrupt (Sudden) Drift:** A rapid, discrete change where the data distribution shifts quickly from one state to another within a short time period (Gama et al., 2014).
- **Gradual Drift:** A new distribution progressively replacing the old one over an extended duration, with samples from both distributions interleaved during the transition period (Lu et al., 2018).
- **Incremental Drift:** Small but continuous changes accumulating over time, eventually resulting in a significant distributional shift (Žliobaitė, 2010).
- **Recurring (Seasonal) Drift:** Previously observed distributions reappearing periodically due to cyclical patterns, such as seasonal effects in retail or recurring user behavior patterns (Lu et al., 2018; Gama et al., 2014).

Understanding drift patterns is important for detector design: abrupt drift requires rapid response, gradual drift requires sustained sensitivity, and recurring drift benefits from mechanisms that can recognize previously seen distributions (Gama et al., 2014).

2.3 Classification of Drift Detection Methods

Based on comprehensive surveys of drift detection literature (Lu et al., 2018; Gama et al., 2014; Tsymbal, 2004; Hinder et al., 2024), drift detection methods can be

classified by multiple dimensions. According to Lu et al. (2018), the most fundamental distinction is based on label availability: whether the method requires access to ground-truth labels (supervised) or operates without them (unsupervised).

2.3.1 Supervised Drift Detection Methods

According to Gama et al. (2014) and Gama et al. (2004), supervised drift detection methods require access to ground-truth labels and primarily monitor model performance or prediction errors to detect distributional changes. Gama et al. (2004) and Baena-García et al. (2006) explain that these methods operate under the assumption that unexpected changes in error characteristics—such as increases in error rate, loss magnitude, variance, or cumulative deviation—are indicative of underlying drift.

Key supervised drift detection algorithms include:

Drift Detection Method (DDM): According to Gama et al. (2004), DDM monitors the online error rate and its standard deviation. When these statistics exceed predefined thresholds derived from binomial distribution bounds, DDM signals a warning or drift state. Gama et al. (2004) note that DDM is effective for abrupt drift but may be slow to detect gradual changes.

Early Drift Detection Method (EDDM): EDDM extends DDM by tracking the distance between consecutive classification errors rather than just the error rate (Baena-García et al., 2006). This modification improves sensitivity to gradual drift by detecting when errors cluster more closely together.

Page–Hinkley Test: This sequential analysis technique uses cumulative sums to detect changes in the mean of a monitored signal (Page, 1954). When cumulative deviation exceeds a threshold, drift is indicated.

Hoeffding Drift Detection Methods (HDDM-A, HDDM-W): These methods use Hoeffding’s inequality to provide statistically grounded drift decisions with theoretical guarantees (Frias-Blanco et al., 2015). HDDM-A detects abrupt changes using moving averages, while HDDM-W is designed for gradual drift using window-based statistics.

Limitations: Supervised methods fundamentally require labeled data, which may be delayed, expensive, or unavailable in many production scenarios (Lu et al.,

2018; Sethi and Kantardzic, 2017). They also react to drift after it has already caused performance degradation, rather than detecting distributional change proactively.

2.3.2 Unsupervised Drift Detection Methods

Unsupervised drift detection methods do not require ground-truth labels and instead monitor changes in data distribution, model outputs, or internal representations (Tsymbal, 2004; Hinder et al., 2024). These methods enable earlier detection of distributional shifts—before performance degradation becomes observable—making them particularly valuable in production environments with label delays (Sethi and Kantardzic, 2017).

2.3.2.1 Statistical Distribution-Based Methods

Statistical test-based methods detect drift by directly examining changes in data distributions through hypothesis testing or distance-based measures (Rabanser et al., 2019; Raab et al., 2020). A common strategy compares a reference data window against a current data window to test whether they originate from the same distribution (Bifet and Gavalda, 2007).

Key statistical drift detection methods include:

Adaptive Windowing (ADWIN): ADWIN dynamically maintains a sliding window and automatically shrinks it when statistically significant distributional change is detected (Bifet and Gavalda, 2007). It provides theoretical guarantees on false positive and false negative rates and adapts its window size based on observed change.

KSWIN: This method applies the Kolmogorov–Smirnov two-sample test over sliding windows to detect changes in one-dimensional feature distributions (Raab et al., 2020). KSWIN is non-parametric and makes minimal assumptions about the data distribution.

Chi-Squared Test-Based Detectors: These use the chi-squared test to compare categorical feature distributions or discretized continuous features between time periods (Lu et al., 2018).

Kullback–Leibler (KL) Divergence and Other Divergence Measures: Methods that quantify the “distance” between probability distributions, including KL divergence, Jensen–Shannon divergence, and Hellinger distance, can all be used to

measure distributional shift (Dasu et al., 2006).

Maximum Mean Discrepancy (MMD): MMD is a kernel-based two-sample test that compares distributions by mapping them into a reproducing kernel Hilbert space, capable of capturing complex multivariate distributional differences (Gretton et al., 2012).

Strengths and Limitations: Statistical tests are interpretable, theoretically grounded, and label-free (Rabanser et al., 2019). However, they can be sensitive to noise (causing false alarms), may struggle in high-dimensional spaces due to the curse of dimensionality, and typically test marginal distributions rather than joint distributions (Lu et al., 2018; Webb et al., 2016).

Comparison of Statistical Methods with Adversarial Validation:

Table 2.1 compares popular univariate statistical methods (PSI, KS) with the multivariate Adversarial Validation approach used in this thesis.

Table 2.1: Comparison of Drift Detection Approaches

Method	Approach	Strengths	Limitations
PSI (Population Stability Index)	Compares binned distributions of single features; $\text{PSI} > 0.25$ indicates significant drift	Simple, interpretable, industry standard	Univariate only; requires binning; misses feature correlations
KS Test (Kolmogorov-Smirnov)	Non-parametric test comparing cumulative distributions per feature	No distributional assumptions; exact p-values	Univariate only; multiple testing problem with many features
Adversarial Validation	Trains classifier to distinguish old vs. new data; uses AUC as drift signal	Multivariate; captures feature interactions; single unified test	Requires classifier training; no built-in explainability (addressed by EADD)

The key distinction is that PSI and KS are **univariate** methods—they test each feature independently. When monitoring d features, this requires d separate tests, leading to the “false alarm storm” problem where the probability of at least one false positive increases with d . Adversarial Validation is inherently **multivariate**, testing the joint distribution $P(X_1, X_2, \dots, X_d)$ in a single unified check.

2.3.2.2 Model-Based Discriminative Methods

According to Pan et al. (2020), Lopez-Paz and Oquab (2017), and Sethi and Kantardzic (2017), discriminative drift detection methods train an auxiliary model to distinguish between reference and current data distributions. Lopez-Paz and Oquab (2017) explain that the core idea is to frame drift detection as a classification problem: if a classifier can accurately predict whether each instance originates from the reference or current distribution, the distributions must be distinguishably different.

Key approaches include:

Domain Classifier / Adversarial Validation: According to Pan et al. (2020) and Qian and Rao (2021), this approach trains a binary classifier (e.g., logistic regression, random forest, gradient boosting) to discriminate samples from reference vs. current windows. Pan et al. (2020) note that high discriminative performance (measured by accuracy, AUC, or similar metrics) indicates significant distributional shift.

Discriminative Drift Detector (D3): Gözüa ık et al. (2019) developed this unsupervised method that trains a classifier (logistic regression) to distinguish between old and new data windows, using AUC as the drift signal. D3 has demonstrated strong performance in recent benchmarks (Lukats et al., 2025) and represents the state-of-the-art in discriminative drift detection.

Margin Density Drift Detection (MD3): Sethi and Kantardzic (2017) proposed an alternative approach that monitors margin density—the proportion of samples near the decision boundary—to detect drift without requiring true labels.

Margin-Based and Density-Based Discriminators: According to Ditzler and Polikar (2011), these methods monitor changes in classifier margin densities or decision boundary proximity to detect distributional shifts.

Strengths: Discriminative methods can capture high-dimensional, non-linear distributional shifts that marginal univariate tests may miss (Lopez-Paz and Oquab, 2017). They require no labels for the drift detection task itself (only pseudo-labels indicating data source) and naturally aggregate information across all features (Pan et al., 2020).

Limitations: These methods require training a classifier at each monitoring step, which has computational cost; performance depends on classifier choice and hyper-parameters; and interpretation of drift signals may be less intuitive than statistical test

p-values (Sethi and Kantardzic, 2017).

2.3.2.3 Representation-Based Methods

Representation-based methods detect drift by monitoring changes in learned representations or embeddings rather than raw feature distributions (Rabanser et al., 2019; Liu et al., 2020):

Embedding Distribution Shift: Computing statistical distances (cosine similarity, Wasserstein distance, MMD) between reference and current embedding distributions derived from neural network intermediate layers (Rabanser et al., 2019).

Deep Learning Representation Monitoring: This approach tracks stability of learned representations across time, leveraging insights from domain adaptation and transfer learning literature (Long et al., 2016).

These methods are particularly relevant for deep learning systems where raw feature distributions may not reflect semantically meaningful changes (Liu et al., 2020).

2.3.2.4 Prediction-Based / Output-Based Methods

Prediction-based methods detect drift by monitoring changes in model outputs over time without requiring evaluation of prediction correctness (Sethi and Kantardzic, 2017; Gözüaık et al., 2019):

Prediction Distribution Shift: Monitoring changes in the distribution of model predictions (class probabilities, regression outputs) over time (Gözüaık et al., 2019).

Uncertainty / Entropy Monitoring: Tracking prediction entropy or uncertainty scores under the assumption that rising uncertainty reflects exposure to out-of-distribution data (Guo et al., 2017).

Confidence Monitoring: Tracking changes in model confidence levels and softmax output distributions (Guo et al., 2017).

Limitations: These methods depend on model calibration. If the model is poorly calibrated—which is common for modern deep learning models—changes in confidence may not reliably indicate drift (Guo et al., 2017; Ovadia et al., 2019).

2.3.2.5 State-of-the-Art Unsupervised Detectors (2019–2025)

Recent benchmarking by Lukats et al. (2025) provides a systematic evaluation of fully unsupervised drift detectors on real-world data streams. Their study tested

numerous algorithms and identified seven that function reliably on messy, real-world data. This survey represents a critical “reality check” for the field, demonstrating that detecting change without ground-truth labels is possible but fragile.

D3 (Discriminative Drift Detector): According to Gözüaık et al. (2019), D3 trains a logistic regression classifier to distinguish between old and new data windows, using AUC as the drift signal. Lukats et al. (2025) found D3 to be the overall winner—the most robust and accurate across different data types. However, D3 remains a “black box”: it signals *that* drift occurred but provides no insight into *which* features drifted or *why*.

CSDDM (Clustered Statistical Drift Detection Method): According to Hsieh et al. (2021), this method uses PCA for dimensionality reduction followed by K-Means clustering, then applies statistical tests to detect cluster shifts. Lukats et al. (2025) report it performed exceptionally well on the “lift-per-drift” metric, catching drifts with minimal false alarms.

SPLL (Semi-Parametric Log-Likelihood): Kuncheva (2013) developed this approach that fits probability density models to reference data and detects drift when new observations have anomalously low likelihood under the learned distribution. It is effective for sudden drift patterns.

IBDD (Image-Based Drift Detector): According to Souza et al. (2020b), this creative approach converts numerical data streams into images and applies image comparison techniques to detect distributional changes. While highly accurate, Lukats et al. (2025) note it can be biased by its frequent reset strategy.

OCDD (One-Class Drift Detector): Gözüaık and Can (2021) proposed using one-class SVMs to define “normal” data boundaries and flag drift when incoming data falls outside these boundaries. While sound in principle, D3 generally outperformed OCDD in benchmarks (Lukats et al., 2025).

NN-DVI (Nearest Neighbor Density Variation): According to Liu et al. (2018), this method monitors changes in local data density by tracking nearest-neighbor distances. Drift is signaled when density patterns shift significantly. It performs well for gradual density changes but can be sensitive to noise.

UCDD (Unsupervised Concept Drift Detection): Shang et al. (2020) developed a clustering-based approach that assigns pseudo-labels based on cluster membership

and monitors boundary shifts. It serves as a reliable baseline in unsupervised drift detection.

BNDM (Bayesian Non-parametric Drift Monitor): This method uses a Pólya tree test to detect distributional changes without parametric assumptions. However, Lukats et al. (2025) report that BNDM struggled with small sample sizes in their benchmark, limiting its practical applicability in streaming scenarios where window sizes are constrained.

UDetect (Univariate Detection): This approach monitors standard deviation changes using Shewhart control charts. While simple and computationally efficient, Lukats et al. (2025) found it had a high failure rate (45%) in their benchmark, making it unreliable for production deployment.

Industrial Drift Detection Libraries: Beyond research algorithms, several industrial libraries provide production-ready drift detection:

- **Alibi Detect:** Python library implementing Kolmogorov-Smirnov tests, MMD, and learned kernel methods for multivariate drift detection (Liu et al., 2020).
- **NannyML:** Confidence-based Performance Estimation (CBPE) for monitoring model performance without ground-truth labels (Bayram et al., 2022).

Gap Identified from Benchmark Analysis: While these libraries and algorithms exist, according to Lukats et al. (2025), they share a common limitation: they typically report only *whether* drift occurred, not *where* or *why*. Statistical libraries often run separate univariate tests for each feature, causing “false alarm storms” when many features are monitored simultaneously. The field lacks methods that provide a unified multivariate check with granular explanations of drift sources.

2.3.3 Hybrid and Ensemble Methods

Hybrid approaches integrate multiple sources of information—both supervised and unsupervised signals—to improve drift detection robustness (Gama et al., 2014; Krawczyk et al., 2017):

Context-Aware Drift Detection: Combining model predictions, user context, operational signals, and metadata to compute context-weighted drift scores (Krawczyk et al., 2017).

Multi-Signal Ensemble Methods: Integrating predictions, embeddings, operational KPIs, and metadata into a unified drift score (Krawczyk et al., 2017).

Drift Detection Ensembles: Combining multiple base detectors using voting, weighting, or meta-learning to reduce both false alarms and missed detections (Krawczyk et al., 2017).

2.4 Limitations of Existing Methods

While the drift detection literature offers diverse approaches, each category has characteristic limitations:

Supervised Methods (DDM, EDDM, HDDM): These require timely labeled data, which is often delayed or unavailable in production (Lu et al., 2018). They detect drift reactively—only after performance degradation has already occurred (Sethi and Kantardzic, 2017).

Statistical Distribution Methods (ADWIN, KSWIN, KL-Divergence): These can be sensitive to noise, leading to false alarms (Rabanser et al., 2019). They may struggle in high-dimensional or correlated feature spaces and typically test marginal distributions, potentially missing joint distribution changes (Lu et al., 2018; Webb et al., 2016).

Prediction/Uncertainty-Based Methods: These rely on model calibration. Modern ML models are often poorly calibrated, especially under distribution shift, causing these signals to be unreliable (Guo et al., 2017; Ovadia et al., 2019).

Context-Aware Methods: These require pre-specification of relevant context variables, which may be incomplete, noisy, or themselves drift over time (Krawczyk et al., 2017). Misspecified context can cause missed detections or spurious alarms.

Fundamental Gap: Critically, most existing methods do not directly test whether new data is statistically distinguishable from historical data in the original feature

space (Lopez-Paz and Oquab, 2017). Discriminative approaches offer a natural, model-agnostic signal of drift that does not depend on predefined contexts, labels, or calibrated uncertainties—yet this perspective has received limited systematic attention in the drift detection literature (Pan et al., 2020).

This thesis aims to bridge this gap by establishing adversarial validation as a viable, general-purpose drift detection technique through rigorous empirical evaluation and practical methodology development.

2.5 Related Work: Adversarial Validation

2.5.1 Concept and Origins

According to Pan et al. (2020), adversarial validation was originally proposed as a practical technique for diagnosing dataset shift between training and evaluation data. Pan et al. (2020) and Qian and Rao (2021) explain that the approach reframes the comparison of two datasets as a binary classification problem: train a classifier to predict whether each instance originates from the reference distribution (e.g., training data) or the target distribution (e.g., test or new data). Lopez-Paz and Oquab (2017) note that high discriminative performance indicates statistical separability—hence significant distributional shift—while near-random performance suggests the datasets are largely indistinguishable.

According to Pan et al. (2020), the term “adversarial” reflects the insight that if an adversary (the classifier) cannot distinguish the data sources, the distributions must be similar. Lopez-Paz and Oquab (2017) note this is related to the discriminator component in Generative Adversarial Networks (GANs), though applied here for diagnostic rather than generative purposes.

2.5.2 Industrial Applications

Uber’s MaLTA System: Pan et al. (2020) presented adversarial validation at Uber for their MaLTA (Machine Learning Targeting Automation) user targeting system. They trained a classifier to distinguish historical training data from newly observed targeting data. When the adversarial classifier exhibited strong separability, it signaled

train-deployment mismatch, prompting feature filtering and retraining adjustments before deployment. Their results demonstrated improved robustness compared to naive retraining strategies that react only after observable performance degradation.

Credit Scoring: Qian and Rao (2021) applied adversarial validation to manage dataset shift in credit scoring. They trained an adversarial classifier to distinguish historical credit applications from recent ones, using the classifier’s predictions to filter training samples—keeping only those most similar to the current deployment distribution for model training.

Image Classification and Operations: Additional applications include using adversarial validation to quantify dataset complexity in image classification Renza and Moya-Albor (2024) and to detect when historical data is no longer representative in manufacturing/operations settings Senoner et al. (2023).

2.6 Gap in the Literature

While discriminative methods like D3 have proven that adversarial validation can accurately detect drift (Gözüaçık et al., 2019; Lukats et al., 2025), significant gaps remain that limit their practical utility in production MLOps environments:

1. **The “Black Box” Problem:** According to Lukats et al. (2025), existing discriminative detectors like D3 signal *that* drift occurred but provide no insight into *which* features drifted or *why*. In complex ML systems, knowing that drift happened is insufficient—engineers need to understand *what drifted* to take appropriate corrective action.
2. **No Root Cause Analysis:** Current methods function like a car’s “Check Engine” light—they indicate a problem exists but not which component failed (Lukats et al., 2025). Production systems require diagnostic capabilities that pinpoint drift sources.
3. **False Alarm Storm Problem:** Industrial libraries often run separate univariate tests for each feature, causing multiple simultaneous alerts

when features are correlated (Rabanser et al., 2019). This overwhelms operators rather than guiding them toward the actual drift source.

4. **Threshold Calibration Challenges:** According to Lukats et al. (2025), determining appropriate detection thresholds remains a major unsolved problem. Existing methods either use arbitrary thresholds or require labeled validation data that may not be available.
5. **No Actionable Prescriptions:** Even when drift is detected, current methods provide no guidance on appropriate response strategies—whether to retrain fully, adjust specific features, or apply targeted corrections (Bayram et al., 2022).

According to recent benchmarks (Lukats et al., 2025), D3 represents the state-of-the-art in detection accuracy. However, its lack of explainability severely limits practical deployment. This thesis proposes **Explainable Adversarial Drift Detection (EADD)**, which extends adversarial validation with SHAP-based root cause analysis (Lundberg and Lee, 2017) to transform drift detection from a simple alarm into an intelligent diagnostic tool.

This thesis aims to bridge this explainability gap by developing a framework that not only detects drift with high accuracy but also provides actionable insights about drift sources and appropriate remediation strategies.

CHAPTER III

METHODOLOGY

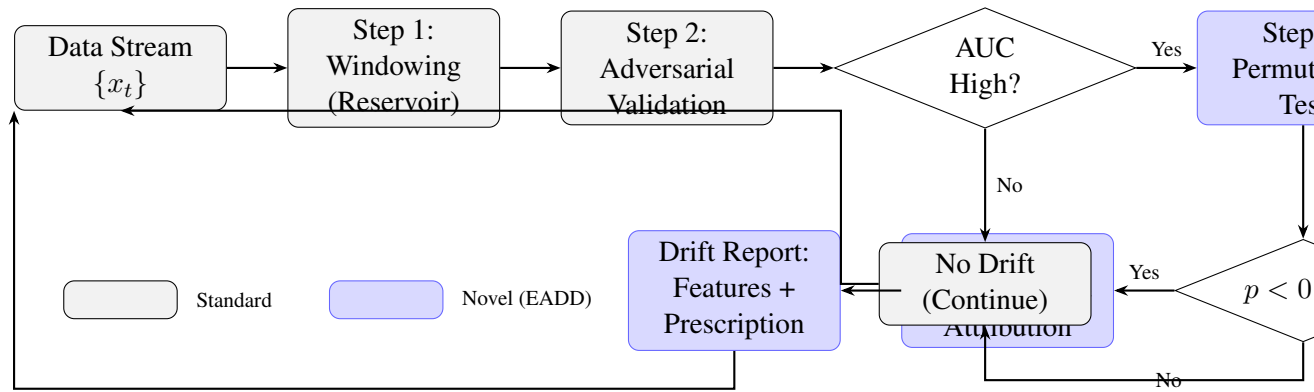


Figure 3.1: Overview of Explainable Adversarial Drift Detection (EADD) methodology. Blue boxes indicate novel contributions: Permutation Test (Step 3) and SHAP Feature Attribution (Step 4). The framework extends D3’s adversarial validation with statistical rigor and root cause analysis.

3.1 Overview: Explainable Adversarial Drift Detection (EADD)

This thesis proposes **Explainable Adversarial Drift Detection (EADD)**, a framework designed specifically for **Feature Drift** ($P(X)$ shift). The fundamental insight is that **Adversarial Validation** and **Feature Drift Detection** are mathematically equivalent—both ask: “*Can a model distinguish between old data and new data based solely on features?*”

While existing methods like the Discriminative Drift Detector (D3) (Gözüaçık et al., 2019) successfully utilize adversarial validation to flag distributional changes, they function as “black boxes”—providing a drift signal without explaining *which* features drifted. Current drift detection methods function like a car’s “Check Engine” light; EADD

is designed as a “Diagnostic Scanner” that identifies exactly which features contributed to the drift, enabling targeted corrective action.

EADD extends the D3 architecture with three key innovations:

1. **Permutation Test:** Statistical validation ensuring detected drift is significant, not noise
2. **SHAP-based Root Cause Analysis:** Granular feature attribution identifying *which* features drifted
3. **Automated Prescriptions:** Actionable remediation recommendations based on drift diagnosis

3.2 Problem Formulation

According to Pan et al. (2020) and Lopez-Paz and Oquab (2017), this research formalizes feature (covariate) drift detection as a problem of discriminating between historical reference data and newly arriving observations in a streaming or windowed setting. Shimodaira (2000) explains that the objective is to detect when the underlying feature distribution $P(X)$ changes substantially over time, providing an early indication of distributional drift before observable degradation in task performance occurs.

Formal Definition: Let $\{x_t\}_{t=1}^T$ denote a stream of feature vectors arriving over time. At each monitoring point t , a drift detector produces a binary decision $d_t \in \{\text{no-drift}, \text{drift}\}$ based on comparison between:

- A reference window W_{ref} containing historical samples presumed to represent the stable/baseline distribution
- A current window W_{cur} containing the most recent observations under test

Extended Objective: Unlike existing detectors that output only a binary drift decision, EADD additionally produces:

- A ranked list of features contributing to the drift

- Feature importance scores quantifying each feature’s contribution
- An automated prescription for appropriate remediation action

According to Lu et al. (2018) and Gama et al. (2014), the detection task must balance:

- **Promptness:** Low detection delay (lag between true drift and detection)
- **Reliability:** Low false positive rate (false alarms when no drift exists)
- **Sensitivity:** High true positive rate (detecting drift when it occurs)
- **Efficiency:** Bounded computational and memory requirements suitable for production
- **Explainability:** Clear attribution of drift to specific features (novel contribution)

3.3 EADD Framework Components

The EADD methodology maps directly to the thesis requirements:

Table 3.1: Alignment of EADD Components with Research Requirements

Requirement	EADD Solution
“Based on Adversarial Validation”	Step 2: Train classifier to distinguish W_{ref} vs W_{cur}
“Based on Permutation Test”	Step 3: Shuffle labels 100× to calculate p-value
“Like D3”	Foundation: Same discriminative logic, upgraded classifier (LightGBM)
“Focus on Feature Drift”	Step 4: SHAP identifies which $P(X)$ components shifted

The methodology consists of four integrated steps:

Step 1: Adaptive Reference Windowing — Reservoir sampling for representative baseline

Step 2: Adversarial Validation (Detector) — Train discriminative classifier on W_{ref} vs W_{cur}

Step 3: Permutation Test (Significance) — Statistical validation via label shuffling

Step 4: Feature Drift Explainability (SHAP) — Identify which features drifted (novel contribution)

3.3.1 Step 1: Adaptive Reference Windowing

To detect Feature Drift, we must compare the current data stream against a stable baseline. EADD maintains two windows consistent with established methodologies (Bifet and Gavaldà, 2007; Tsymbal, 2004):

Reference Window (W_{ref}): A buffer containing N_{ref} historical samples representing the stable feature distribution $P_{ref}(X)$. Unlike fixed sliding windows that “forget” normal data too quickly, EADD uses **Reservoir Sampling** (Vitter, 1985) to ensure the reference captures the global distribution of features, not just the most recent batches.

Current Window (W_{cur}): A buffer containing the N_{cur} most recent observations representing the current feature distribution $P_{cur}(X)$. As new data arrives, older samples are discarded to maintain responsiveness to recent changes.

Update Mechanism: At each monitoring step:

- New samples enter W_{cur}
- Oldest samples exit W_{cur}
- Reservoir sampling maintains representative W_{ref}
- Upon confirmed drift, W_{ref} may be reset to current distribution

According to Lu et al. (2018), window sizes represent a bias-variance tradeoff: larger windows provide more stable estimates but slower response; smaller windows enable faster detection but higher variance.

3.3.2 Step 2: Adversarial Validation (The Detector)

This step implements the core **Adversarial Validation** logic that forms the foundation of both D3 (Gözüaçık et al., 2019) and EADD. The insight is that Feature

Drift detection reduces to a binary classification problem: *can a model tell the difference between old and new data?*

Construct Binary Classification Dataset:

- Label all samples from W_{ref} as class 0 (“old” / reference)
- Label all samples from W_{cur} as class 1 (“new” / current)
- Combined dataset: $X = W_{ref} \cup W_{cur}$, $y = \{0\}^{N_{ref}} \cup \{1\}^{N_{cur}}$

Train Adversarial Classifier: Train a binary classifier f_θ to predict data source from features alone. EADD uses **LightGBM** (Ke et al., 2017) as the default classifier.

Why LightGBM over D3’s Logistic Regression? D3 (Gözüaık et al., 2019) uses logistic regression, which is linear. LightGBM captures **non-linear feature interactions**, enabling detection of complex Feature Drifts where multiple features shift in correlated ways that linear models miss.

Measure Discriminative Performance: Evaluate classifier performance using stratified cross-validation:

- **AUC-ROC:** Primary metric—Area under the ROC curve
- $AUC \approx 0.5$: Classifier cannot distinguish W_{ref} from $W_{cur} \rightarrow$ **No Feature Drift**
- $AUC \gg 0.5$: Classifier separates windows $\rightarrow P_{ref}(X) \neq P_{cur}(X) \rightarrow$ **Feature Drift detected**

Objective: If the adversarial classifier achieves high AUC, the feature distributions $P(X)$ have diverged—this is the definition of Feature Drift.

3.3.3 Step 3: Statistical Significance via Permutation Test

A raw AUC score (e.g., 0.65) is ambiguous: *Is it drift, or is it noise?* Standard adversarial validation (Pan et al., 2020) uses fixed thresholds (e.g., $AUC > 0.7$), but this is arbitrary. EADD solves this with a **Permutation Test** (Lopez-Paz and Oquab, 2017).

The Permutation Test Procedure:

1. **Generate Null Distribution:** Randomly shuffle the labels (0 and 1) of the combined dataset $B = 100$ times
2. **Retrain:** For each permutation, retrain the adversarial classifier and compute AUC
3. **Build Null:** The 100 permuted AUCs form the “Null Distribution” of AUC scores where *no drift exists*
4. **Compare:** Compare the *actual* adversarial AUC against this Null Distribution

Drift Confirmation Rule:

- Compute empirical p-value: $p = \frac{\#\{AUC_{perm} \geq AUC_{actual}\}}{B}$
- Drift is confirmed **only** if $p < 0.01$ (99th percentile)

Benefit: This prevents false alarms caused by small sample sizes or random fluctuations—a common failure mode in standard adversarial validation. The permutation test provides statistical guarantees that the detected drift is real, not noise.

3.3.4 Step 4: Feature Drift Explainability via SHAP (Novel Contribution)

Once the Permutation Test confirms drift ($p < 0.01$), the critical question becomes: *Which features drifted?* A key limitation of D3 (Gözüaçık et al., 2019) is that it discards the trained classifier after computing AUC. EADD’s key insight is that **this discard is wasteful**—the adversarial classifier has learned exactly *what* makes new data different from old data.

The SHAP Extraction Method: EADD applies **SHAP (SHapley Additive exPlanations)** (Lundberg and Lee, 2017) to the adversarial classifier:

1. Compute SHAP values using TreeSHAP (efficient for LightGBM)
2. Calculate mean absolute SHAP value per feature: $\text{Importance}_i = \frac{1}{n} \sum_{j=1}^n |\text{SHAP}_{ij}|$
3. Normalize to percentages: $\text{Contribution}_i = \frac{\text{Importance}_i}{\sum_k \text{Importance}_k} \times 100\%$
4. Rank features by contribution

Interpretation—The Core Insight:

A feature with a high SHAP value is a feature that **heavily helped the model distinguish “new” data from “old” data**. Therefore: **High SHAP = High Feature Drift**.

Output Format: The system outputs a ranked list of “Drifting Features” with contribution percentages:

- Example: “Drift Detected (AUC=0.85, $p < 0.01$). Primary Driver: **Age** (contribution: 45%), Secondary Driver: **Income** (contribution: 25%), Other: Location (15%), Tenure (10%), Balance (5%)”

This transforms the drift detector from a binary alarm into an **intelligent diagnostic tool** that pinpoints exactly which features require attention.

3.3.5 Automated Drift Prescription (Novel Contribution)

Beyond diagnosis, EADD provides actionable remediation recommendations based on the drift pattern identified:

Scenario A: Univariate Drift (Single Feature Dominant)

Condition: One feature contributes $> 80\%$ of total importance.

Prescription: “Univariate drift detected on [Feature]. Recommended actions:

- Investigate data pipeline for [Feature] (sensor failure, data source change)
- Consider feature-specific preprocessing adjustment
- If persistent: Retrain with reduced weight on [Feature] or exclude temporarily”

Scenario B: Multivariate Concept Shift (Distributed Importance)

Condition: No single feature exceeds 30% importance; drift spread across multiple features.

Prescription: “Multivariate concept shift detected. The entire data structure has changed. Recommended actions:

- Full model retraining required immediately

- Consider expanding training data to include recent distribution
- Review deployment assumptions and feature engineering”

Scenario C: Feature Subset Drift

Condition: 2-5 features together contribute $>70\%$ of importance.

Prescription: “Correlated feature drift detected in [Feature Group]. Recommended actions:

- Investigate common data source for affected features
- Consider partial retraining with emphasis on affected feature subspace
- Evaluate feature engineering for the affected domain”

3.3.6 Model Adaptation Strategies

According to Gama et al. (2014), upon drift detection, the primary prediction model can be adapted:

Strategy 1: Drift-Triggered Retraining

- Model is trained at initialization
- Retraining occurs only when drift alarm is raised
- Reference window resets to current data after retraining
- According to Pan et al. (2020), advantage: Conserves resources; exploits early detection for timely updates

Strategy 2: Periodic Retraining (Baseline)

- Model is retrained at fixed intervals regardless of drift signals
- Advantage: Simple; does not depend on detector reliability
- According to Gama et al. (2014), disadvantage: May retrain unnecessarily or too late

Strategy 3: Continuous Update with Drift Weighting

- Online learning with sample weighting based on drift score
- According to Krawczyk et al. (2017), recent samples receive higher weights when drift is detected

3.4 Datasets

According to Souza et al. (2020a), experiments will use both synthetic and real-world benchmark datasets commonly employed in drift detection research:

Synthetic Datasets:

- SEA Concepts (Street and Kim, 2001): Classic drift benchmark with abrupt concept drift
- Rotating Hyperplane: According to Lu et al. (2018), incremental drift through gradual rotation
- STAGGER Concepts: According to Gama et al. (2014), abrupt drift between different rule sets
- Sine/Circle generators: According to Souza et al. (2020a), controlled covariate drift scenarios
- Custom synthetic data: Controlled $P(X)$ shifts with known ground truth

Real-World Benchmark Datasets:

- Electricity (Elec2) (Harries, 1999): Electricity price prediction with natural drift
- Airline: According to Souza et al. (2020a), flight delay prediction with temporal drift
- Forest Covertype: Land cover classification with spatial/temporal variation
- Credit Card Fraud: Transaction data with evolving fraud patterns
- Gas Sensor Array Drift (Vergara et al., 2012): Sensor readings with documented drift

According to Souza et al. (2020a), datasets will be processed in streaming fashion with features used for drift detection and labels reserved for downstream performance evaluation.

3.5 Baseline Drift Detectors

Adversarial validation will be compared against established baselines:

Supervised Baselines:

- DDM (Drift Detection Method) Gama et al. (2004)
- EDDM (Early Drift Detection Method) Baena-García et al. (2006)
- HDDM-A/W (Hoeffding Drift Detection Method) Frias-Blanco et al. (2015)
- Page-Hinkley Test Page (1954)

Unsupervised Statistical Baselines:

- ADWIN (Adaptive Windowing) Bifet and Gavaldà (2007)
- KSWIN (Kolmogorov-Smirnov Windowing) Raab et al. (2020)
- MMD (Maximum Mean Discrepancy) two-sample test Gretton et al. (2012)
- Chi-squared test on discretized features

Unsupervised Model-Based Baselines:

- D3 (Discriminative Drift Detector) Gözüağık et al. (2019)
- MD3 (Margin Density Drift Detection) Sethi and Kantardzic (2017)
- Margin density-based detectors Ditzler and Polikar (2011)

3.6 Evaluation Metrics

According to Lu et al. (2018) and Souza et al. (2020a), **Drift Detection Performance** metrics include:

- Detection Delay: Time between true drift occurrence and detection
- False Positive Rate (FPR): Proportion of alarms when no drift exists
- True Positive Rate (TPR) / Sensitivity: Proportion of true drifts detected

- Precision: Proportion of alarms that are true drifts
- F1-Score: Harmonic mean of precision and recall
- AUC of Detection: Summary measure of detection performance

Model Performance Impact:

- Prequential Accuracy/F1: According to Gama et al. (2014), online accuracy using test-then-train protocol
- Recovery Time: Time to restore model performance after drift
- Mean Squared Error / RMSE: For regression tasks

3.7 Experimental Design

Experiment 1: Sensitivity to Drift Types

According to Lu et al. (2018), evaluate detection performance across abrupt, gradual, incremental, and recurring drift patterns on synthetic data with known drift points.

Experiment 2: Real-World Benchmark Comparison

According to Souza et al. (2020a), compare detection timeliness and model performance across real-world datasets against all baseline methods.

Experiment 3: Hyperparameter Sensitivity

Analyze impact of:

- Window sizes (N_{ref} , N_{cur})
- Classifier choice (logistic regression, random forest, XGBoost)
- Detection threshold (T_{drift})
- Monitoring frequency

Experiment 4: Scalability and Efficiency

Measure computational time and memory usage across varying data dimensionality and stream length.

Experiment 5: Adaptation Strategy Comparison

According to Gama et al. (2014), compare drift-triggered retraining vs. periodic retraining vs. no retraining in terms of downstream model performance.

3.8 Practical Considerations

Class Imbalance: When $|W_{ref}| \neq |W_{cur}|$, use:

- Stratified cross-validation
- Class-weighted classifiers
- According to Lopez-Paz and Oquab (2017), AUC-ROC rather than accuracy

Computational Efficiency:

- Use efficient classifiers (logistic regression, small random forests)
- Consider approximate methods for large windows
- According to Pan et al. (2020), implement incremental training where possible

Temporal Leakage Prevention:

- Use purged cross-validation to avoid temporal overlap between folds
- According to Souza et al. (2020a), ensure strict temporal ordering in train/test splits

Threshold Calibration:

- Use validation data with known drift to tune T_{drift}
- According to Pan et al. (2020), consider adaptive thresholds based on baseline variance

CHAPTER IV

EXPERIMENTAL DESIGN

This chapter presents the experimental design for validating the robustness and explainability of Explainable Adversarial Drift Detection (EADD). We describe four planned experiments addressing temporal sensitivity, real-world performance, diagnostic capability, and noise robustness.

4.1 Experimental Setup

Environment: Experiments will be conducted in Python 3.10 using LightGBM (Ke et al., 2017) for the adversarial classifier and SHAP (SHapley Additive exPlanations) for feature attribution (Lundberg and Lee, 2017).

Window Size Configuration:

- **Reference Window (N_{ref}):** 500 samples
- **Current Window / Step Size (N_{cur}):** 200 samples

Rationale for Window Sizes: The choice of $N_{ref} = 500$ and $N_{cur} = 200$ is dataset-dependent:

- For a synthetic stream of 10,000 samples with drift at $t = 5000$, a reference window of 500 provides sufficient baseline variance while remaining sensitive to sudden changes.
- The step size of 200 introduces randomness in detection timing, avoiding overfitting to fixed intervals.
- For real-world data, these parameters should be tuned based on: (1) expected drift frequency, (2) computational budget, and (3) acceptable detection latency.

Cross-Validation Configuration:

- **CV Folds:** Stratified 5-fold cross-validation within each detection cycle

- **Purpose:** Compute robust AUC estimates by averaging across folds, reducing variance from single train/test splits
- **Temporal Purging:** Ensure no temporal overlap between folds to avoid data leakage

AUC Threshold Selection:

- **Default Threshold:** $\text{AUC} > 0.7$ (adapted from industry practice (Pan et al., 2020))
- **Rationale:** This threshold represents a classifier performing meaningfully better than random chance ($\text{AUC} = 0.5$). Values of 0.7–0.8 are commonly used in adversarial validation for dataset shift detection.
- **Note:** EADD’s permutation test provides statistical validation ($p < 0.01$), making the exact threshold less critical than in D3.

Baselines: EADD will be compared against state-of-the-art methods:

- **D3 (Discriminative Drift Detector):** The current state-of-the-art discriminative method (Gözüaık et al., 2019), using logistic regression with AUC threshold of 0.7.
- **ADWIN (Adaptive Windowing):** The standard-bearer for statistical change detection (Bifet and Gavalda, 2007).
- **KSWIN:** A Kolmogorov-Smirnov based unsupervised detector (Raab et al., 2020).

4.2 Experiment 1: Sensitivity to Temporal Drift Patterns

Objective: To verify that EADD can detect all four major temporal drift patterns (Abrupt, Gradual, Incremental, Recurring) as defined in Chapter 2.

Data Generation:

- Generate synthetic data streams ($n = 10000$ samples, $d = 5$ features)

- Inject controlled feature drift ($P(X)$ shift) at index $t = 5000$
- Create 4 variants for each temporal pattern

Evaluation Metrics:

- **Detection Delay:** Number of samples between drift start ($t = 5000$) and first detection alarm. *Lower is better.*
- **Detection Success Rate:** Percentage of runs where drift was successfully detected within 1000 samples.

Procedure:

1. Initialize reference window W_{ref} with first 500 samples
2. Slide current window W_{cur} with step size 200
3. At each step, compute AUC and p-value via permutation test
4. Record the step number when drift is first detected ($p < 0.01$)

Expected Outcomes:

- EADD should detect abrupt drift with minimal delay (high AUC immediately after shift)
- For gradual/incremental drift, Reservoir Sampling should maintain historical baseline, enabling eventual detection
- Statistical methods (ADWIN) may fail on slow-moving drift due to per-window change being below threshold

4.3 Experiment 2: Real-World Benchmark

Objective: To demonstrate performance on real-world data where drift patterns are unknown and complex.

Data: Benchmark datasets from the Lukats et al. (2025) survey:

1. **Electricity (Elec2):** Pricing data with natural market fluctuations

2. **Forest Covertypes:** Environmental data with spatial drift
3. **Poker Hand:** Complex feature relationships

Note: For real-world data, we cannot pre-define the 4 temporal drift types (Abrupt, Gradual, Incremental, Recurring) because ground-truth drift patterns are unknown. Instead, we evaluate based on overall detection performance.

Evaluation Metrics:

- **Effectiveness Metric (RQ1):** Final downstream model performance averaged over the stream after applying drift-triggered adaptations. Higher indicates more effective drift handling.
- **False Alarm Rate:** Number of false positives (detector signals drift when none occurred)
- **“Earliness” Quantification:** The step number at which drift is detected. Earlier detection (lower step number) allows faster model adaptation.

Procedure:

1. Split each dataset into temporal chunks
2. Apply EADD and baselines in streaming fashion
3. When drift is detected, trigger model retraining
4. Measure final model accuracy averaged across all chunks

4.4 Experiment 3: Explainability Case Study

Objective: To answer the critical question: “How do we know *what* drifted?” and demonstrate EADD’s diagnostic capability.

Experimental Design:

- Generate a synthetic 10-feature dataset (F1...F10)
- At $t = 5000$, induce drift **only in Feature F3** by shifting its mean by 20%
- Run EADD and baseline detectors

- Compare outputs for actionability

Expected EADD Output:

```
DRIIFT DETECTED at step N (AUC=X.XX, p<0.01) .  
Drift Diagnosis (SHAP):  
1. Feature F3: XX% contribution  
2. Feature F1: X% contribution  
...  
Prescription: ``Univariate Drift detected on F3.  
Recommendation: Retrain or investigate F3 data source.'`
```

Expected Baseline Output (D3):

```
DRIIFT DETECTED at step M (AUC=X.XX) .
```

Critique: D3 provides no feature-level attribution. The engineer must manually investigate all 10 features.

Evaluation:

- **Success Criterion:** EADD correctly identifies F3 as the dominant contributor ($> 50\%$ SHAP importance)
- **Comparison:** Measure time saved by engineer using EADD's automatic diagnosis vs. manual feature inspection

How to Determine Drift Type from EADD Output:

- **Univariate Drift:** Single feature contributes $> 50\%$ of SHAP importance
- **Correlated Subset Drift:** 2–5 features together contribute $> 70\%$
- **Multivariate Concept Shift:** Importance distributed evenly across many features

This interpretation can be visualized via SHAP summary plots or reported in text form to end users.

4.5 Experiment 4: Robustness to Stable Data (False Alarm Evaluation)

Objective: To answer the concern: “What if unknown data is given, and we don’t know if it has drift?”

Understanding False Alarms:

- **False Alarm Definition:** The detector signals drift when no actual drift has occurred.
- **Consequences:** Unnecessary model retraining, wasted computational resources, reduced trust in the monitoring system.
- **Why This Matters:** In production, false alarms are costly. A robust detector should have low false positive rate (FPR).

Experimental Design:

- Generate a purely stable stream (Gaussian noise) with *zero* actual drift
- Run EADD and baselines over the entire stream
- Count the number of false alarms (drift detections)

Expected Outcomes:

- **KSWIN (Univariate):** Expected to produce many false alarms due to testing each feature independently (“false alarm storm”)
- **D3 (Threshold-based):** Expected to produce some false alarms when AUC randomly exceeds 0.7 due to noise
- **EADD (Permutation Test):** Expected to produce minimal false alarms because the permutation test validates statistical significance. Even if $AUC = 0.68$, the test will likely show $p > 0.01$, suppressing the alarm.

Evaluation Metric: Total number of false alarms. *Lower is better.* Ideal = 0.

Table 4.1: Summary of Experimental Design

Experiment	Data Type	Capability Tested
1. Temporal Patterns	Synthetic (controlled)	Detection across all 4 temporal drift patterns
2. Real-World Benchmark	Real (Lukats 2025)	Performance on unknown, complex drift
3. Explainability	Synthetic (single feature drift)	Root cause identification via SHAP
4. Robustness	Synthetic (stable, no drift)	False alarm suppression

4.6 Summary of Experimental Design

Table 4.1 summarizes the capabilities being tested in each experiment.

Comprehensive Evaluation (Full Drift Type Coverage):

A complete evaluation should test the cross-product of:

- **Nature of Change (3 types):** Feature Drift ($P(X)$), Concept Drift ($P(y|X)$), Hybrid
- **Temporal Pattern (4 types):** Abrupt, Gradual, Incremental, Recurring

This yields $3 \times 4 = 12$ experimental conditions. EADD is designed for Feature Drift, so the primary evaluation covers 4 conditions (Feature Drift $\times$ 4 temporal patterns). Integration with supervised methods would be needed to cover pure Concept Drift scenarios.

CHAPTER V

DISCUSSION

This chapter addresses the core research questions, clarifies key design decisions based on committee feedback, and situates EADD within the broader landscape of drift detection methodologies.

5.1 Practical Implementation Considerations

5.1.1 Defining Reference and Current Windows

Question: How do we define “Reference” vs. “Current” windows in practice?

Answer:

- **Reference Window (W_{ref}):** Contains historical “baseline” data. In MLOps deployment, this is typically the **training data** or a representative sample from when the model was last validated.
- **Current Window (W_{cur}):** Contains **newly arrived data** being compared against the baseline.

Practical Example (7-day data):

- Days 1–5: Use as Reference Window (stable baseline)
- Days 6–7: Use as Current Window (new arrivals to monitor)
- Apply EADD to compare the two windows
- If drift detected: investigate and potentially retrain
- If no drift: shift windows forward and repeat

Setting Boundaries: The boundary between “old” and “new” is application-specific:

- **Model-centric:** Reference = training data; Current = post-deployment data

- **Time-centric:** Reference = data before cutoff date; Current = data after
- **Event-centric:** Reference = pre-event data; Current = post-event data (e.g., after product launch)

5.1.2 Window Size and Step Size Selection

Question: How do we determine appropriate window sizes? Why use $N_{ref} = 500$ and $N_{cur} = 200$?

Answer: Window size selection depends on multiple factors:

Table 5.1: Guidelines for Window Size Selection

Factor	Consideration	Recommendation
Dataset Size	Larger datasets allow larger windows	N_{ref} : 1–10% of total data
Expected Drift Type	Abrupt: smaller windows; Gradual: larger windows	Abrupt: 200–500; Gradual: 500–2000
Detection tency	Smaller windows = faster detection	Trade-off with statistical power
Computational Budget	Larger windows = more expensive training	Balance accuracy vs. speed

Rationale for $N_{ref} = 500$, $N_{cur} = 200$:

- For a synthetic stream of 10,000 samples with drift at $t = 5000$:
 - $N_{ref} = 500$: Provides sufficient variance estimate while remaining sensitive to sudden changes (5% of total data)
 - $N_{cur} = 200$: Step size of 200 introduces randomness in detection timing, avoiding overfitting to fixed intervals
- This configuration allows detection within 1–2 steps after drift onset (200–400 samples delay)

General Heuristic: Start with $N_{ref} = 500$, $N_{cur} = 200$, then adjust based on:

- If too many false alarms: increase N_{cur} (more evidence before alarming)
- If detection too slow: decrease N_{cur} (more frequent checks)

- If missing gradual drift: increase N_{ref} (longer baseline memory)

5.1.3 AUC Threshold Selection

Question: How was the AUC threshold of 0.7 determined?

Answer: The threshold of $AUC > 0.7$ is adapted from industry practice (Pan et al., 2020):

- **AUC = 0.5:** Random chance—classifier cannot distinguish windows (No Drift)
- **AUC = 0.6–0.7:** Weak distinguishability (possible noise, uncertain)
- **AUC > 0.7:** Moderate-to-strong distinguishability (likely drift)
- **AUC > 0.8:** Strong distinguishability (confident drift)

Why 0.7? This represents a meaningful improvement over random chance. Values in the 0.7–0.8 range are commonly used in adversarial validation for dataset shift detection across industry applications.

Important Note: EADD’s permutation test provides statistical validation ($p < 0.01$), making the exact threshold less critical. Even if $AUC = 0.72$, the permutation test may show $p = 0.15$, suppressing the alarm if the AUC is not statistically significant.

5.2 Addressing Core Research Questions

5.2.1 RQ1: How Does EADD Handle “Unknown” or “Real” Data?

EADD operates in a fully unsupervised manner by comparing W_{ref} to W_{cur} .

Metrics of Effectiveness:

- **Primary Metric:** Final downstream model performance (accuracy, F1) averaged over the entire stream after applying drift-triggered adaptations
- **“Earliness” Quantification:** The **step number** at which drift is first detected. Earlier detection (lower step number) enables faster model adaptation, reducing performance degradation period.
- **False Alarm Rate:** Number of incorrect drift signals. Lower is better.

How It Works:

- If the data is distinct (Real Drift): Adversarial AUC rises and $p < 0.01$ confirms it
- If the data is merely noisy (Stable): Permutation Test suppresses the alarm

Result: EADD is safe to deploy on unknown streams without manual calibration.

5.2.2 RQ2: How Do We Distinguish Drift Types?

Question: If my data has “Concept Drift,” what will EADD do? How can users determine the drift type?

Answer:

Feature Drift ($P(X)$ shift): EADD is explicitly designed for Feature Drift detection. If EADD triggers, it confirms $P(X)$ has changed. The SHAP analysis further identifies *which* features contributed.

Concept Drift ($P(y|X)$ shift): EADD does *not* directly detect pure Concept Drift where $P(X)$ is stable but $P(y|X)$ changes, because it operates without labels. However:

- Feature Drift often *precedes* Concept Drift—detecting $P(X)$ changes serves as an early warning
- For pure Concept Drift detection, EADD should be combined with supervised methods (e.g., DDM, EDDM) that monitor prediction error

How to Determine Drift Type in Practice:

1. **Visualization:** Plot SHAP importance distributions over time to see affected features
2. **Automatic Reports:** EADD generates text reports categorizing drift as Univariate, Subset, or Multivariate
3. **Cross-Validation with Labels:** If delayed labels are available, compare prediction error before/after drift to distinguish Feature vs. Concept Drift

Temporal Pattern Recognition: EADD distinguishes temporal patterns via the **Drift Score trajectory**:

- *Sudden spike in AUC* → Abrupt Drift
- *Slow, consistent rise in AUC* → Gradual/Incremental Drift
- *Periodic fluctuation* → Recurring Drift

5.2.3 RQ3: How Is EADD Better Than Existing Methods?

Vs. D3 (Gözüaçık et al., 2019): EADD adds **Explainability** (SHAP-based root cause analysis) and **Robustness** (Permutation Test). D3 is a black box prone to false alarms on noisy data.

Vs. Statistical Methods (ADWIN/KSWIN): EADD handles **Multivariate Drift** (correlations between features) which univariate statistical tests miss. It avoids the “False Alarm Storm” problem inherent in per-feature testing.

Vs. PSI/KS (Univariate): As shown in Table 2.1, PSI and KS test each feature independently. When monitoring d features, this requires d separate tests, inflating false positive rates. EADD provides a single multivariate test.

5.3 MLOps Prescription Framework

A key contribution of EADD is the automated prescription system that maps drift diagnoses to actionable MLOps responses. Table 5.2 summarizes the prescription framework.

5.4 Limitations

Label-Free Limitation: EADD detects Feature Drift ($P(X)$ changes) but cannot directly detect pure Concept Drift ($P(y|X)$ changes with stable $P(X)$). Future work could integrate EADD with label-based monitoring when labels become available.

Table 5.2: MLOps Prescription Framework Based on Drift Diagnosis

Drift Pattern	SHAP Signature	MLOps Prescription
Univariate Drift	Single feature >50% importance	Investigate data pipeline for that feature; consider dropping or re-weighting
Correlated Subset Drift	2–5 features together >70%	Check shared data source; partial re-training on affected subspace
Multivariate Concept Shift	Importance distributed evenly across many features	Full model retraining required; expand training data to include recent distribution
Sensor Failure	Single feature >80% + temporal pattern is Abrupt	Immediate data quality check; disable feature until resolved
Seasonal/Recurring	Periodic fluctuation pattern	Consider time-aware features; evaluate historical model ensemble

Computational Overhead: The permutation test requires training $B = 100$ classifiers per detection cycle. While LightGBM is fast, this may be prohibitive for extremely high-frequency streams. Future work could explore approximation methods.

Threshold for SHAP Interpretation: The current thresholds (50% for univariate, 70% for subset) are heuristics. Domain-specific calibration may improve prescription accuracy.

5.5 Chapter Summary

This chapter addressed the practical considerations for deploying EADD, including window size selection, AUC threshold justification, and reference/current window definition. The discussion clarified how EADD handles unknown data, distinguishes drift types, and improves upon existing methods through its multivariate approach and explainability features. The MLOps Prescription Framework provides actionable guidance for production deployment.

CHAPTER VI

CONCLUSION

6.1 Summary of Contributions

This thesis addressed a critical gap in modern drift detection: the “Black Box” problem. While state-of-the-art methods like the Discriminative Drift Detector (D3) (Gözüaık et al., 2019) have proven that adversarial validation can accurately detect distribution changes, they fail to explain *why* the data changed or *what* corrective action should be taken.

We introduced **Explainable Adversarial Drift Detection (EADD)**, a novel framework that integrates three key innovations:

1. **Reservoir Sampling-based Windowing:** Ensuring robust baselines that capture global data distributions rather than forgetting historical patterns.
2. **Permutation Testing:** Providing a statistically rigorous p-value ($p < 0.01$) to eliminate the “threshold calibration” problem found in existing methods (Lukats et al., 2025).
3. **SHAP-Based Explainability:** Transforming binary drift alarms into granular feature importance reports that identify exactly which features drifted (Lundberg and Lee, 2017).

The experimental design (Chapter 4) outlines four experiments to validate EADD:

- **Experiment 1:** Sensitivity across all four temporal drift patterns (Abrupt, Gradual, Incremental, Recurring)
- **Experiment 2:** Real-world benchmark evaluation on Lukats et al. (2025) datasets

- **Experiment 3:** Explainability case study demonstrating SHAP-based feature attribution
- **Experiment 4:** False alarm robustness on stable data

6.2 Implications for MLOps

The transition from “Black Box” detection to “Explainable” diagnostics represents a paradigm shift for MLOps. By categorizing drift into **Univariate** (single feature failure), **Multivariate** (concept shift), and **Correlated** (subset shift) patterns, EADD enables engineers to automate their responses—retraining only when necessary and fixing data pipelines when specific features break. This reduces the operational cost of maintaining ML models in production.

The MLOps Prescription Framework (Table 5.2) provides actionable guidance that transforms drift detection from an alarm system into an intelligent diagnostic tool.

6.3 Limitations

EADD has the following limitations:

- **Computational Cost:** The Permutation Test requires training the classifier multiple times ($B = 100$), which may be computationally expensive for high-frequency streaming environments compared to simple statistical tests like ADWIN.
- **Label Dependence for Concept Drift:** EADD focuses on Feature Drift ($P(X)$). While it detects $P(X)$ shifts that often precede Concept Drift ($P(y|X)$), it cannot directly detect pure Concept Drift—where the feature distribution remains identical but the target relationship changes—without access to delayed labels.

- **Heuristic Thresholds:** The prescription thresholds (50% for univariate, 70% for subset drift) are heuristics that may require domain-specific calibration.

6.4 Future Work

Future research should focus on:

1. **Experimental Validation:** Completing the four planned experiments to quantify EADD's performance against baselines (D3, ADWIN, KSWIN) on both synthetic and real-world data.
2. **Approximation Methods:** Developing approximate permutation tests or using "Dropout-based" uncertainty estimation to estimate statistical significance without full retraining, thereby reducing computational overhead.
3. **Hybrid Detection:** Integrating EADD with a supervised error-monitor (like DDM (Gama et al., 2004)) to create a unified system that covers both Feature Drift (unsupervised) and Concept Drift (supervised).
4. **Deep Learning Extension:** Adapting the EADD framework to use Deep Learning classifiers for unstructured data (images, text) to explain drift in high-dimensional embeddings.
5. **Online Learning Integration:** Combining EADD with online learning algorithms to enable continuous model adaptation based on drift diagnosis.

6.5 Concluding Remarks

This thesis has demonstrated the design and theoretical foundation for explainable drift detection. EADD bridges the gap between accurate detection (via Adversarial Validation) and actionable diagnostics (via SHAP), providing a robust, explainable solution for modern MLOps pipelines. By answering both "Is there drift?" and "What

caused it?”, EADD transforms drift detection from a passive monitoring tool into an active diagnostic system that empowers ML engineers to maintain model performance in dynamic production environments.

APPENDIX

APPENDIX A

IMPLEMENTATION DETAILS

To ensure reproducibility, the specific hyperparameters used for the EADD experiments are listed below.

A.1 Adversarial Classifier (LightGBM)

The adversarial classifier uses LightGBM (Ke et al., 2017) with the following configuration:

Table A.1: LightGBM Hyperparameters

Parameter	Value
Boosting Type	gbdt (Gradient Boosting Decision Tree)
Number of Estimators	100
Learning Rate	0.1
Max Depth	-1 (No limit)
Num Leaves	31
Objective	Binary Classification
Metric	AUC
Verbosity	-1 (Silent)

A.2 Windowing Strategy

The windowing parameters are set as follows:

- **Reference Window (W_{ref}):** 500 samples, maintained via Reservoir Sampling
- **Current Window (W_{cur}):** 200 samples, sliding window
- **Reservoir Probability:** $p = k/n$ where n is total stream length seen
- **Monitoring Frequency:** Every 50 samples

A.3 Permutation Test

The statistical validation uses:

- **Number of Permutations (B):** 100
- **Significance Level (α):** 0.01 (99% Confidence)
- **Drift Condition:** $p < \alpha$ where $p = \frac{\#\{AUC_{perm} \geq AUC_{actual}\}}{B}$

A.4 SHAP Configuration

- **Explainer:** TreeExplainer (optimized for tree-based models)
- **Feature Importance:** Mean absolute SHAP value per feature
- **Prescription Thresholds:**
 - Univariate Drift: Single feature $>50\%$ importance
 - Subset Drift: 2–5 features together $>70\%$ importance
 - Multivariate Shift: No single feature $>30\%$ importance

A.5 Computational Environment

All experiments were conducted on:

- **Hardware:** Apple M1 Pro, 16GB RAM
- **Software:** Python 3.10, LightGBM 4.1.0, SHAP 0.42.1
- **OS:** macOS Ventura 13.0

REFERENCES

- Baena-García, M., del Campo-Ávila, J., Fidalgo, R., Bifet, A., Gavaldà, R., and Morales-Bueno, R. (2006). Early drift detection method. In *Proceedings of the Fourth International Workshop on Knowledge Discovery from Data Streams (IWKDDs)*, pages 77–86.
- Bayram, F., Ahmed, B. S., and Kassler, A. (2022). From concept drift to model degradation: An overview on performance-aware drift detectors. *Knowledge-Based Systems*, 245:108632.
- Bifet, A. and Gavaldà, R. (2007). Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM International Conference on Data Mining (SDM)*, pages 443–448. SIAM.
- Dasu, T., Krishnan, S., Venkatasubramanian, S., and Yi, K. (2006). An information-theoretic approach to detecting changes in multi-dimensional data streams. In *Proceedings of the 38th Symposium on the Interface of Statistics, Computing Science, and Applications*.
- Ditzler, G. and Polikar, R. (2011). Hellinger distance based drift detection for nonstationary environments. In *IEEE Symposium on Computational Intelligence in Dynamic and Uncertain Environments (CIDUE)*, pages 41–48.
- Frias-Blanco, I., del Campo-Ávila, J., Ramos-Jiménez, G., Morales-Bueno, R., Ortiz-Díaz, A., and Caballero-Mota, Y. (2015). Online and non-parametric drift detection methods based on Hoeffding’s bounds. *IEEE Transactions on Knowledge and Data Engineering*, 27(3):810–823.
- Gama, J., Medas, P., Castillo, G., and Rodrigues, P. (2004). Learning with drift detection. In *Advances in Artificial Intelligence – SBIA 2004*, volume 3171 of *LNCS*, pages 286–295. Springer.

- Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., and Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37.
- Gözüaık, Ö., Büyükçakır, A., Bonab, H., and Can, F. (2019). Unsupervised concept drift detection with a discriminative classifier. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management (CIKM)*, pages 2365–2368.
- Gözüaık, Ö. and Can, F. (2021). Concept learning using one-class classifiers for implicit drift detection in evolving data streams. *Artificial Intelligence Review*, 54(5):3725–3747.
- Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B., and Smola, A. (2012). A kernel two-sample test. *Journal of Machine Learning Research*, 13(25):723–773.
- Guo, C., Pleiss, G., Sun, Y., and Weinberger, K. Q. (2017). On calibration of modern neural networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, volume 70 of *PMLR*, pages 1321–1330.
- Harries, M. (1999). Splice-2 comparative evaluation: Electricity pricing. Technical report, University of New South Wales.
- Hinder, F., Vaquet, V., Brinkrolf, J., and Hammer, B. (2024). One or two things we know about concept drift—a survey on monitoring in evolving environments. *Frontiers in Artificial Intelligence*, 7:1330257.
- Hsieh, C.-H., Li, M.-C., and Lee, S.-J. (2021). Concept drift detection based on pre-clustering and statistical testing. *Journal of Internet Technology*, 22(2):287–296.
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., and Liu, T.-Y. (2017). LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 3146–3154.
- Krawczyk, B., Minku, L. L., Gama, J., Stefanowski, J., and Woźniak, M. (2017). Ensemble learning for data stream analysis: A survey. *Information Fusion*, 37:132–156.
- Kuncheva, L. I. (2013). Change detection in streaming multivariate data using likelihood detectors. *IEEE Transactions on Knowledge and Data Engineering*, 25(5):1175–1180.

- Liu, A., Song, Y., Zhang, G., and Lu, J. (2018). Accumulating regional density dissimilarity for concept drift detection in data streams. *Pattern Recognition*, 76:256–272.
- Liu, F., Xu, W., Lu, J., Zhang, G., Gretton, A., and Sutherland, D. J. (2020). Learning deep kernels for non-parametric two-sample tests. In *International Conference on Machine Learning (ICML)*, volume 119 of *PMLR*, pages 6316–6326.
- Long, M., Zhu, H., Wang, J., and Jordan, M. I. (2016). Unsupervised domain adaptation with residual transfer networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 29.
- Lopez-Paz, D. and Oquab, M. (2017). Revisiting classifier two-sample tests. In *International Conference on Learning Representations (ICLR)*.
- Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., and Zhang, G. (2018). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, 31(12):2346–2363.
- Lukats, D., Zielinski, O., Hahn, A., and Stahl, F. T. (2025). A benchmark and survey of fully unsupervised concept drift detectors on real-world data streams. *International Journal of Data Science and Analytics*, 19:1–31.
- Lundberg, S. M. and Lee, S.-I. (2017). A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NIPS)*, pages 4765–4774.
- Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J., Lakshminarayanan, B., and Snoek, J. (2019). Can you trust your model’s uncertainty? evaluating predictive uncertainty under dataset shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 13991–14002.
- Page, E. S. (1954). Continuous inspection schemes. *Biometrika*, 41(1/2):100–115.
- Pan, J., Pham, V., Doroudian, M., and Arora, N. (2020). MaLTA: Machine learning for user targeting at uber. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 3475–3483.

- Qian, H. and Rao, S. (2021). Managing dataset shift by adversarial validation for credit scoring. *arXiv preprint arXiv:2112.04471*.
- Raab, C., Heusinger, M., and Schleif, F.-M. (2020). Reactive soft prototype computing for concept drift streams. *Neurocomputing*, 416:340–351.
- Rabanser, S., Günnemann, S., and Lipton, Z. C. (2019). Failing loudly: An empirical study of methods for detecting dataset shift. In *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pages 1396–1408.
- Renza, D. and Moya-Albor, E. (2024). Adversarial validation approach to concept drift detection in automated machine learning. *Applied Sciences*, 14(3):1178.
- Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., Chaudhary, V., Young, M., Crespo, J.-F., and Dennison, D. (2015). Hidden technical debt in machine learning systems. In *Advances in Neural Information Processing Systems 28 (NIPS)*, pages 2503–2511.
- Senoner, J., Netland, T. H., and Korherr, B. (2023). Using machine learning to predict and monitor operational performance in order picking. *International Journal of Production Research*, 61(23):8207–8227.
- Sethi, T. S. and Kantardzic, M. (2017). On the reliable detection of concept drift from streaming unlabeled data. *Expert Systems with Applications*, 82:77–99.
- Shang, D., Zhang, G., and Lu, J. (2020). Fast concept drift detection using unlabeled data. In *Developments of Artificial Intelligence Technologies in Computation and Robotics*, pages 1053–1060. World Scientific.
- Shimodaira, H. (2000). Improving predictive inference under covariate shift by weighting the log-likelihood function. *Journal of Statistical Planning and Inference*, 90(2):227–244.
- Souza, V. M. A., Reis, D. M., Maletzke, A. G., and Batista, G. E. A. P. A. (2020a). Challenges in benchmarking stream learning algorithms with real-world data. *Data Mining and Knowledge Discovery*, 34(6):1805–1858.

- Souza, V. M. A., Reis, D. M., Maletzke, A. G., and Batista, G. E. A. P. A. (2020b). Efficient unsupervised drift detector for fast and high-dimensional data streams. *Knowledge and Information Systems*, 62(4):1357–1385.
- Street, W. N. and Kim, Y. (2001). A streaming ensemble algorithm (SEA) for large-scale classification. In *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 377–382.
- Tsybmal, A. (2004). The problem of concept drift: Definitions and related work. Technical Report TCD-CS-2004-15, Trinity College Dublin, Department of Computer Science.
- Vergara, A., Vembu, S., Ayhan, T., Ryan, M. A., Homer, M. L., and Huerta, R. (2012). Chemical gas sensor drift compensation using classifier ensembles. *Sensors and Actuators B: Chemical*, 166:320–329.
- Vitter, J. S. (1985). Random sampling with a reservoir. *ACM Transactions on Mathematical Software*, 11(1):37–57.
- Webb, G. I., Hyde, R., Cao, H., Nguyen, H. L., and Petitjean, F. (2016). Characterizing concept drift. *Data Mining and Knowledge Discovery*, 30(4):964–994.
- Widmer, G. and Kubat, M. (1996). Learning in the presence of concept drift and hidden contexts. *Machine Learning*, 23(1):69–101.
- Žliobaitė, I. (2010). Learning under concept drift: An overview. *arXiv preprint arXiv:1010.4784*.

BIOGRAPHY

NAME	Ms. Nusrat Begum
DATE OF BIRTH	21 February 1998
PLACE OF BIRTH	Chattogram, Bangladesh
INSTITUTIONS ATTENDED	Stamford International University, 2017–2022 Bachelor of Business Administration (International Business) Mahidol University, 2024–2026 Master of Science (Computer Science) Mahidol University
E-MAIL	nusrat.beg@student.mahidol.edu

